Music Recommendation System

Project Owner: Willian Franco da Nóbrega

Table of Contents

Business Problem

Nowadays we have many recommendation algorithms everywhere, specially on streaming platforms, deciding for us what we are going to watch or hear next. The problem is often, either by the politics of the platforms or by the set up of those algorithms, we end up getting the same thing recommended over and over again, regardless of what you asked for.

For our business problem, we are dealing with music streaming platforms, where many users claim to end up listening to the same random songs over and over again, one example is when they use the playlist generation feature from those platforms and the songs have nothing to do with what they asked for.

To address this issue, the following algorithm was developed, aiming to provide music lovers with playlists where they can explore similar songs to the one they like, or ask for a playlist that simply match the vibe they are in.

Scope and Preparation

The scope of this project is to create a feature-based recommendation system that analyzes song lyrics using language processing to suggest similar music or find songs by mood/theme.

The main idea is to be able to recommend songs identifying the theme using the respective lyrics and genre of the song.

In preparation for this project, we needed to gather the following data:

1- Dataset of song tracks in a personal apple music library.
2- Lyrics of all songs in this library (using Genius API).

Intermediate Steps

Cleaning was necessary when using the data, such as:

3- Removing from the lyrics:
    - Stopwords
    - Sentences inside [], i.E: [Intro]
4- Lemmatization in English songs.
5- Stemming (optional) in all songs where language are available in our stemming library.
6- Creation of matching keys to match the information between lyrics folder and tracks dataset.
7- One hot encoding genre columns.

Besides cleaning, we had the calculation of:

8- Semantic Analysis
9- Sentiment Analysis
10- Embedding
11- Cosine similarity

Testing

When tested, our system successfully created playlist based on the emotions and genre, as per follow:

12- Similar Song search:

```python
recommend_song(
    "...Baby One More Time",
    top_n=20
)
```

✓ 0.2s Python

	artist	song	Genre	sentiment	# similarity
219	Ashley Tisdale	What If	Pop	neutral	0.8346730975722438
1740	Leah Kate	Cry Baby	Pop	neutral	0.8232799794493009
1969	Christian Leave	Please Notice	Pop	neutral	0.8227572249134403
2132	Leyla Blue	What A Shame	Pop	negative	0.8187618169765984
881	Leah Kate	Boy Next Door	Pop	neutral	0.8077727774397573
302	Zara Larsson	Stick With You	Pop	neutral	0.8051782090625905
1558	Beyoncé	Love On Top	Pop	neutral	0.8013644202998075
85	DUDA BEAT	BABY	Pop	neutral	0.7969372262055423
1828	Ariana Grande	Into You	Pop	neutral	0.7955225654073071
2045	Beyoncé	Why Don't You Love Me	Pop	neutral	0.7935482338518662

20 rows x 5 cols 10 per page Page 1 of 2

13-Semantic Search:

```python
semantic_search(
    "songs about heartbreak and still missing someone",
    genre="Alternative"
)
```

✓ 0.6s Python

	artist	song	Genre	language	sentiment
1863	Taylor Swift	A Message From Taylor	Pop	en	neutral
1774	Taylor Swift	hoax	Alternative	en	neutral
1651	Gal Gosta	O Amor Em Paz	Latin Jazz	pt	negative
616	Marina Sena	SENSEI	Pop	pt	neutral
545	Adele	Someone Like You	Pop	en	neutral
513	Flávio Venturini	Pra lembrar de nós	Brazilian	pt	neutral
727	half•alive	still feel.	Alternative	en	neutral
1192	Marisa Monte	Ainda Bem	MPB	pt	neutral
1428	Madonna	Fragile	Pop	en	negative
1135	Taylor Swift	marjorie	Alternative	en	neutral

10 rows x 6 cols 10 per page Page 1 of 1

Evaluation

For this step of the project, it was not possible to evaluate the results as we need the feedback from users to understand and help us tune it down the algorithm.

Currently, it is scheduled for the next steps the following implementation:

1- Get the query from the user
2- Generate the playlist
3- Get the feedback from the user: like or dislike
4- Give the feedback from the user to all songs in the playlist as: “Query”: “1(Like)”, “Song_ID”
5- Store the feedback with the context (query)
6- Collect all feedback from users with the same/similar query
7- Calculate the mean for the songs, in this context.

Example:

Song "Tears" for query "melancholic":

- Appeared in 4 recommendation sets

- Received: 3 LIKES + 1 DISLIKE

- Score: 3/4 = 0.75 (75% positive)

Song "Someone Like You" for query "melancholic":

- Appeared in 4 recommendation sets

- Received: 3 LIKES + 1 DISLIKE

- Score: 3/4 = 0.75 (75% positive)

Song "Happy" for query "melancholic":

- Appeared in 4 recommendation sets

- Received: 0 LIKES + 4 DISLIKES

- Score: 0/4 = 0.00 (0% positive) BAD

Results

We have a recommendation system that:

- Offer songs based on similar songs in our library
- Offer songs based on context, phrases inputted by the user
- Gives the opportunity for users to explore their libraries without being limited by streaming platforms algorithms that are often based on capital
- Give small artists the chance to be heard more often.

We are solving our business problem by offering users the chance of having their request heard and used to improve our recommendations.

Future Steps

Besides enhancing our recommendations with the user feedback, in the future scope we aim for connecting with users libraries easier so we can give more customized results;

Having a web interface to make it easier for usage;

Creating the playlists in your streaming platform automatically;

References

To make this project possible, we used the repository from: https://github.com/b5i/apple-playlist-lyrics-downloader

With this repository as an inspiration, I adapted to read our apple library dataset and search the songs in the GENIUS API to download each lyrics txt file.